

AUTOMATED DEVICE TESTING USING SELENIUM, BEHAVE AND QEMU

PROJECT REPORT

Submitted By

AKASH SAHA

Department Name

CSE AIML

Institution Name

INSTITUTE OF ENGINEERING AND MANAGEMENT

Academic Year

2023–2027

Technology Stack

Python • Selenium WebDriver • Behave BDD • Gherkin • QEMU • Mock Device Application

1. Abstract

This project presents an automated testing framework for a simulated device/application environment using Python, Selenium WebDriver, Behave, and QEMU. The framework combines browser automation with Behavior-Driven Development (BDD) so that test requirements can be represented as readable scenarios and then executed through Python step definitions. A mock device application provides a controlled target, while QEMU is used as the virtualized environment layer. The project is designed to demonstrate a repeatable, modular, and maintainable approach to automated testing without requiring continuous access to physical device hardware.

2. Introduction

Software testing is an important stage of the software development lifecycle. Manual testing can become repetitive, time-consuming, and difficult to reproduce consistently when the same scenarios have to be executed many times. Test automation addresses these issues by allowing predefined test cases to be executed programmatically. This project develops a structured automation solution using Selenium for web interaction, Behave for BDD-based test organization, Python for implementation, and QEMU for virtualization or emulation of the target environment.

3. Background

Selenium WebDriver is widely used for automating browser-based applications. Behave is a Python BDD framework that executes scenarios written using Gherkin syntax. BDD connects requirements with executable tests through a readable Given–When–Then structure. QEMU provides virtualization and emulation capabilities and can be used to reproduce a controlled environment. In this project, these technologies are integrated with a mock application to demonstrate an end-to-end automated testing workflow.

4. Problem Statement

Testing device-oriented applications can involve physical hardware, repeated environment configuration, and manual execution of test cases. Physical devices may not always be available and reproducing the same environment for every run can be difficult. The project addresses this challenge by providing a controlled mock application and a virtualized testing environment, together with automated browser testing and BDD-based test specifications.

5. Objectives

The objectives of the project are to automate application interactions using Selenium WebDriver; organize requirements and test cases using Behave and Gherkin; separate test specifications from their Python implementations; provide a reproducible virtualized environment using QEMU; reduce repetitive manual testing; improve test maintainability; and establish a foundation that can later be integrated with continuous testing or CI/CD workflows.

6. Scope

The project focuses on automated testing of a simulated device/application workflow. The scope includes feature definition, Python step implementation, test setup and teardown, browser automation, application validation, and virtual environment startup. The project does not attempt to reproduce every physical characteristic of real hardware. Instead, it provides a practical framework for demonstrating automated functional testing in a controlled environment.

7. Technologies and Tools

Python: Used to implement automation logic, Behave steps, hooks, and supporting application components.

Selenium WebDriver: Used to automate browser interactions and validate web-based application behavior.

Behave: Provides the BDD test framework and executes Gherkin scenarios.
Gherkin: Provides a readable syntax for describing features and expected behavior.
QEMU: Provides virtualization/emulation for a reproducible target environment.
Mock Device Application: Acts as a controlled simulated target for testing.
Python Virtual Environment: Isolates project dependencies.

8. System Architecture

The system follows a layered architecture. The feature layer contains human-readable test requirements. The step-definition layer converts those requirements into executable Python operations. Selenium provides browser interaction, while the mock application acts as the target. The QEMU layer provides the virtualized environment when required. Environment hooks coordinate setup and cleanup across the test lifecycle.

9. Project Directory Structure

```
Selenium_Behave_QEMU_Project/  
├──  
│   ├── .venv/  
│   ├──  
│   │   ├── features/  
│   │   │   ├── device.feature  
│   │   │   ├── environment.py  
│   │   │   ├── steps/  
│   │   │   └── device_steps.py  
│   │   ├──  
│   │   ├── mock_device/  
│   │   │   └── app.py  
│   │   ├──  
│   │   ├── qemu/  
│   │   │   └── start_qemu.bat
```

10. Module Description

Module / File	Description
device.feature	Defines the test feature and scenarios using Gherkin syntax. It describes expected behavior in a readable form.
environment.py	Contains Behave lifecycle hooks used for setup, initialization, cleanup, and management of test resources.
device_steps.py	Contains Python step definitions that implement the actions described in the feature file.
mock_device/app.py	Provides the simulated device/application used as the controlled test target.
qemu/start_qemu.bat	Windows batch script used to launch the QEMU environment according to the project's configuration.
.venv/	Project-specific Python virtual environment used to isolate dependencies.

11. Behavior-Driven Development

Behavior-Driven Development focuses on describing software behavior in terms that are understandable to both technical and non-technical stakeholders. Behave implements this approach in Python. A feature contains one or more scenarios, and each scenario is expressed using Given, When, and Then statements. This separation improves readability and allows the same step implementations to be reused across multiple scenarios.

12. Gherkin Structure

Feature: describes the functionality under test.

Scenario: describes one specific behavior or test case.

Given: establishes the initial state or precondition.

When: represents an action or event.

Then: represents the expected outcome.

And / But: extends a previous step while keeping the scenario readable.

13. Selenium Automation

Selenium WebDriver is responsible for interacting with the browser. Depending on the scenario, the automation can open a page, locate elements, enter data, click controls, wait for dynamic elements, read displayed information, and verify expected results. Selenium operations are implemented in Python step definitions rather than being embedded directly into the feature description.

14. QEMU Integration

QEMU is used as the virtualization/emulation component of the project. A QEMU startup script provides a repeatable mechanism for launching the required environment. Virtualization is useful for testing because it can reduce dependence on physical devices and make environment initialization more consistent. The precise machine image, networking configuration, and QEMU command-line options depend on the local project setup.

15. Mock Device Application

The mock device application provides a controlled testing target. It allows the automation workflow to be developed and demonstrated without requiring the actual physical device. A mock target also makes repeated testing easier because the application can be restarted and returned to a known state.

16. Test Execution Workflow

The overall workflow begins by preparing the Python environment and starting the required mock application or QEMU environment. Behave then reads the feature file and maps each Gherkin step to its corresponding Python implementation. Selenium performs browser actions where required. The resulting application state is checked against the expected behavior. Finally, environment hooks perform cleanup and release resources.

17. Execution Commands

```
Change to the project directory:  
cd /d D:\Selenium_Behave_QEMU_Project
```

```
Activate the virtual environment:  
.venv\Scripts\activate
```

```
Execute the Behave suite:  
behave
```

```
Execute with verbose output:  
behave -v
```

18. Test Validation

Validation is performed by comparing the actual application state with the expected state defined in the scenario. Assertions can verify displayed text, element visibility, application status, state changes, or other measurable outcomes. A failed assertion causes the corresponding scenario to be reported as failed, making unexpected behavior visible in the test results.

19. Test Environment Management

The `environment.py` file can be used to centralize lifecycle operations. This includes starting or configuring the browser, preparing test resources, collecting diagnostic information, and closing resources after execution. Centralizing these operations reduces duplication and keeps individual step definitions focused on the behavior being tested.

20. Synchronization and Reliability

Automated browser tests must account for page loading and dynamic content. Explicit waits are generally preferable to arbitrary fixed delays because they wait for a specific condition. Stable locators, appropriate exception handling, controlled test data, and reliable cleanup also contribute to test stability.

21. Advantages

The framework provides readable test scenarios, reusable step definitions, separation between requirements and implementation, repeatable execution, reduced dependence on physical hardware, and a modular project structure. The use of QEMU and a mock application also makes environment preparation more controllable during development and testing.

22. Limitations

The mock environment cannot represent every characteristic of real hardware. QEMU configuration may require platform-specific setup. Selenium tests depend on compatible browser and WebDriver versions. Device-specific hardware behavior, physical sensors, electrical conditions, and other hardware characteristics may require separate testing on real equipment.

23. Future Enhancements

Future development can include the Page Object Model for improved maintainability, automated screenshots and logs, HTML or Allure reporting, parallel execution, cross-browser testing, CI/CD integration, Docker-based test environments, test-data management, and more comprehensive QEMU lifecycle automation.

24. Conclusion

The Selenium–Behave–QEMU project demonstrates a structured approach to automated testing by integrating browser automation, BDD, Python, virtualization, and a controlled mock application. Behave provides readable test specifications, Selenium executes browser interactions, Python implements the automation logic, and QEMU supports a reproducible virtual environment. The resulting framework provides a practical foundation for functional automation and can be extended toward larger automated testing and continuous integration systems.

25. References

1. Selenium WebDriver documentation and Python Selenium resources.
2. Behave documentation and Gherkin language reference.
3. QEMU documentation and virtualization resources.
4. Python documentation for virtual environments and automation development.